

Voting Classifier: Hard and Soft Voting

Contents

1. Overview	2
2. The core idea, on decision surfaces	2
3. Hard voting and soft voting	3
3.1 Hard voting	3
3.2 Soft voting	3
3.3 When the two disagree	4
3.4 Which one to use	5
4. VotingClassifier on the iris data	6
4.1 Making the problem hard	6
4.2 Each base model alone	6
4.3 Hard and soft voting	7
4.4 Weights	7
5. One algorithm, different settings	8
6. Summary	10
7. Key terms	10

Where this fits: Pipeline map steps 8 (Model), 9 (Evaluate). See the [Course map](#).



- **Builds on:** Independent and mutually exclusive events ([Note 84](#)); Ensemble learning ([Note 101](#)).
- **Leads to:** Grid and random search ([Note 106](#)).
- **Compare with:** Train-test split ([Note 13](#)); Bagging ([Note 105](#)); Stacking and blending (Video 127, coming).

1. Overview

Key point: A voting classifier combines several classifiers trained on the same data. Hard voting counts their predicted labels; soft voting averages their predicted probabilities. Weights let some models count more.

The idea and the probability behind voting are in the [voting ensemble Note](#). This Note applies them to classification:

- a demo of decision surfaces, showing what voting does to a boundary;
- the two kinds of voting, **hard** and **soft**, and how they differ;
- scikit-learn's `VotingClassifier` on a real dataset, with its `weights` hyperparameter;
- voting over one algorithm with different settings.

The Notebook ([notebook.ipynb](#)) runs every experiment. The Dash app `app.py` lets us pick a dataset, base models and the voting type, and redraws every decision surface.

2. The core idea, on decision surfaces

Key point: Each base model draws its own boundary; the voting classifier's boundary mixes them, and often generalises better than each one.

Recall the setup: base models M_1 , M_2 , M_3 are trained on the same data D . For a query point x_q , each predicts a class (0 or 1 for a two-class problem), and the voting classifier returns the most common one, the **mode** of the predictions.

A decision surface (the [KNN Note](#), section 5) shows this best. The app `app.py` offers six toy datasets with two input columns, such as concentric circles, a U shape, two spirals and XOR, and five base models: KNN, logistic regression, Gaussian naive Bayes, SVM and random forest (a forest of trees, covered in later Notes). It trains the chosen models and the voting classifier on 80% of the points and reports accuracy on the other 20%.

On the concentric-circles data, one class forms rings around the other, so no straight line separates them:

Base models (hard voting)	Each model's test accuracy	Voting classifier
logistic regression, Gaussian naive Bayes	0.53, 0.60	0.63
logistic regression, SVM	0.53, 0.87	0.77
logistic regression, Gaussian naive Bayes, random forest	0.53, 0.60, 0.89	0.66

Logistic regression draws a straight line, which cannot follow rings, so it scores barely better than guessing. Naive Bayes draws a curved boundary and does a little better. Voting with both gives 0.63, above each of them: the vote's boundary mixes the line and the curve.

With more models, the vote's boundary takes something from each. With few test points (here 100), accuracies jump around, so the shape of the boundary is often a better guide than the last digit of the score: a smooth boundary that follows the data tends to generalise better.

The vote does **not** always win. Adding the strong random forest (0.89) to two weak models gives a hard vote of only 0.66: the two weak models outvote the good one. In such a case we simply use the random forest alone. Soft voting, below, rescues this case.

3. Hard voting and soft voting

Key point: Hard voting takes the most common predicted label. Soft voting averages the predicted probabilities of each class and picks the highest average.

Almost every classification algorithm can give its answer as probabilities: for example, class 0 with probability 0.6 and class 1 with probability 0.4. The predicted label is the class with the highest probability.

3.1 Hard voting

Key point: Each model casts one vote for its predicted label; the label with most votes wins.

Hard voting is what we have done so far: each model's predicted label counts as one vote, and the majority wins. It is scikit-learn's default, `voting="hard"`.

Take two models on a two-class problem. M1 gives (class 0: 0.6, class 1: 0.4) and predicts 0; M2 gives (0.8, 0.2) and also predicts 0. Hard voting: 0 and 0, so the answer is **0**.

3.2 Soft voting

Key point: Average each class's probability across the models; the class with the highest average wins.

Soft voting (`voting="soft"`) uses the probabilities themselves.

1. **In words:** for each class, average the probability that the models give it; predict the class with the largest average.
2. **Formula:** with n models, where model i gives class c the probability $p_i(c)$,

$$\bar{p}(c) = \frac{1}{n} \sum_{i=1}^n p_i(c), \quad \hat{y} = \text{the class } c \text{ with the largest } \bar{p}(c)$$

3. **Example:** the same two models:

$$\bar{p}(0) = \frac{0.6 + 0.8}{2} = 0.7, \quad \bar{p}(1) = \frac{0.4 + 0.2}{2} = 0.3$$

so the answer is again **0**.

The same works for more classes. Five classifiers on a three-class problem give these probabilities (in %):

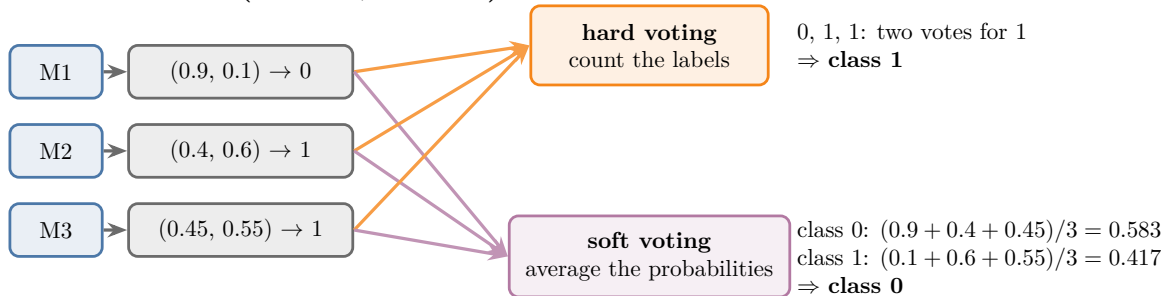
Classifier	Class 1	Class 2	Class 3
C1	90	8	2
C2	20	70	10
C3	10	60	30
C4	30	65	5
C5	25	55	20
Average	35	51.6	13.4

Class 2 has the highest average, so soft voting predicts **class 2**. Hard voting agrees here: four of the five classifiers pick class 2.

3.3 When the two disagree

Key point: Soft voting listens to how sure each model is, so one very confident model can outweigh two unsure ones.

probabilities of (class 0, class 1)



M2 and M3 barely prefer class 1; M1 is very sure of class 0. Soft voting weighs how sure each model is.

Figure 1: Hard and soft voting reach different answers when one model is very sure and the others are not

Extra: In the examples above both kinds of voting agree. Figure 1 shows a case where they do not. M1 is very sure of class 0 (0.9); M2 and M3 lean only slightly to class 1 (0.6 and 0.55). Hard voting counts two votes for class 1 and answers **1**. Soft voting averages 0.583 for class 0 against 0.417 for class 1 and answers **0**.

3.4 Which one to use

Key point: Soft voting is often better, but not always. The voting type is a hyperparameter: try both.

Soft voting often does better, because a probability carries more information than a bare label. On the concentric circles with logistic regression, naive Bayes and random forest (Figure 2):

- the base models score 0.53, 0.60 and 0.89;
- **hard** voting scores **0.66**: the two weak models outvote the forest;
- **soft** voting scores **0.92**, better than every base model.

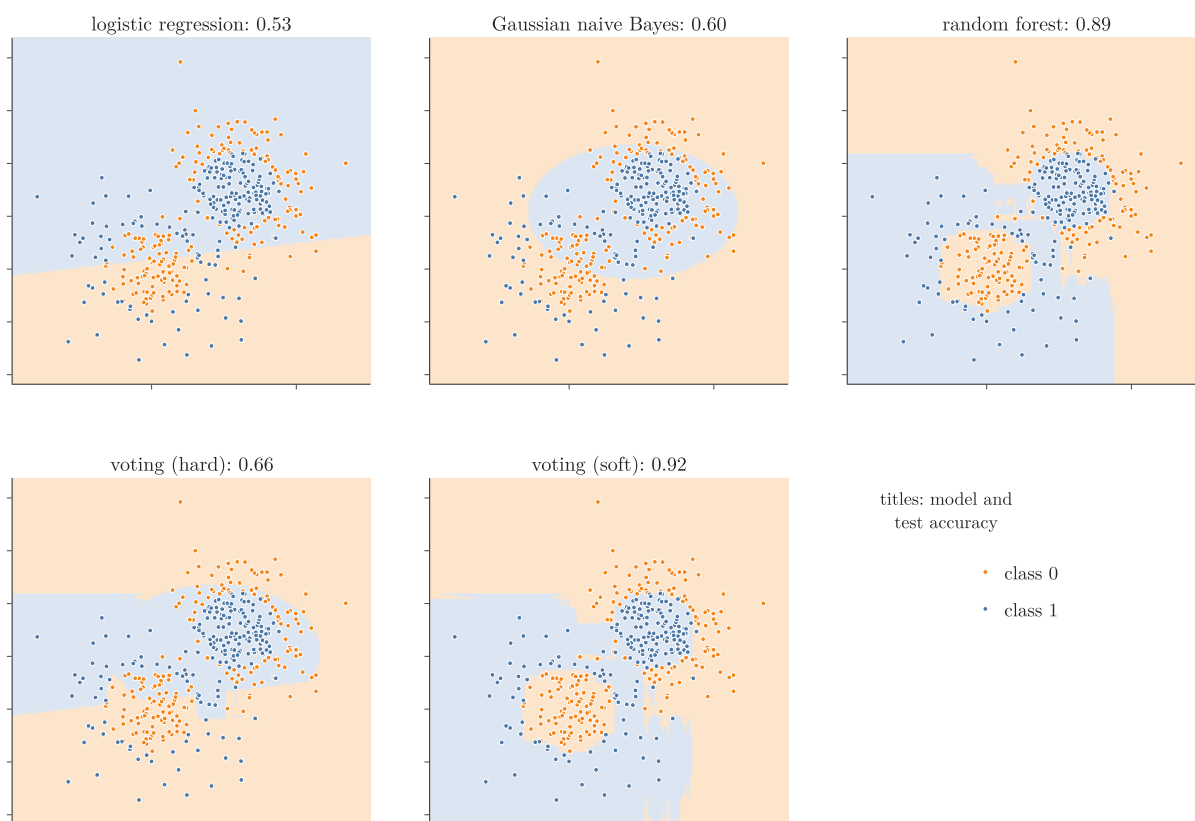


Figure 2: Decision surfaces on the concentric-circles data: three base models, then hard and soft voting. Titles give test accuracy

Soft voting also tends to give a smoother decision surface, and that smoothing is often why it does better. It is not a rule, though: on the iris data in section 4, soft voting does slightly

worse. The voting type is a hyperparameter like any other, so we try both and keep the better.

Extra: Soft voting needs every base model to give probabilities, through a `predict_proba` method. Most classifiers have one. An SVM does not by default: older code wrote `SVC(probability=True)`, which is deprecated in scikit-learn 1.9. The current way is to wrap the SVM: `CalibratedClassifierCV(SVC(), ensemble=False)`.

4. VotingClassifier on the iris data

Key point: On a hard two-species iris problem, logistic regression alone (0.75) beats both voting classifiers (0.67 hard, 0.64 soft); weights lift the vote to 0.71.

4.1 Making the problem hard

Key point: Keep only the two species that overlap and only the two sepal columns.

The iris data (the [softmax regression Note](#)) is easy: most algorithms reach 98 to 99% accuracy. To make it hard, we keep only the two sepal columns (length and width) and only versicolor and virginica. In these two columns the two species overlap heavily, while setosa sits apart. That leaves 100 rows.

Python: Building the harder problem.

```
from sklearn.datasets import load_iris

df = load_iris(as_frame=True).frame
df = df.rename(columns={"target": "species"})
# drop setosa (species 0); keep sepals and label
new_df = df[df["species"] != 0][
    ["sepal length (cm)", "sepal width (cm)", "species"]]
X = new_df.iloc[:, 0:2]
y = new_df.iloc[:, -1]
```

4.2 Each base model alone

Key point: Logistic regression scores 0.75, random forest 0.60 and KNN 0.61.

We use three base models: logistic regression, random forest and KNN. Each is scored with 10-fold cross-validation (the [pipelines Note](#), section 8): the data is cut into 10 parts, the model is trained on 9 and tested on the tenth, ten times, and we average the 10 accuracies.

Python: Scoring each base model.

```
from sklearn.model_selection import cross_val_score

estimators = [("lr", LogisticRegression()),
              ("rf", RandomForestClassifier(random_state=42)),
```

```

        ("knn", KNeighborsClassifier())]
for name, model in estimators:
    scores = cross_val_score(model, X, y, cv=10,
                             scoring="accuracy")
    print(name, scores.mean())    # lr 0.75, rf 0.60, knn 0.61
estimators is a list of tuples: each tuple is (a name as a string, the model object).
VotingClassifier takes exactly this format.

```

4.3 Hard and soft voting

Key point: Hard voting 0.67, soft voting 0.64: here the best single model wins.

Python: The voting classifier.

```

from sklearn.ensemble import VotingClassifier

vc = VotingClassifier(estimators=estimators, voting="hard")
cross_val_score(vc, X, y, cv=10).mean()    # 0.67

vc = VotingClassifier(estimators=estimators, voting="soft")
cross_val_score(vc, X, y, cv=10).mean()    # 0.64
voting is "hard" by default. fit trains every base model; predict combines them.

```

Both votes beat random forest and KNN but not logistic regression (0.75). This happens: the two weaker models outvote the best one. The honest choice here is logistic regression alone.

4.4 Weights

Key point: `weights` gives each model's vote a different importance. Giving logistic regression weight 3 lifts soft voting from 0.64 to 0.71.

By default every model's vote counts the same, as in a democracy. The `weights` hyperparameter changes that: `weights=[5, 1]` makes the first model's vote count five times as much as the second's. In hard voting it multiplies the votes; in soft voting it gives a weighted average of the probabilities.

1. **In words:** each model's probability is multiplied by its weight, and the sum is divided by the total weight.
2. **Formula:** with weights $w_1, \dots, w_n$,

$$\bar{p}(c) = \frac{\sum_{i=1}^n w_i p_i(c)}{\sum_{i=1}^n w_i}$$

3. **Example:** in Figure 1, give M1 weight 2 and the others weight 1. For class 1:

$$\bar{p}(1) = \frac{2(0.1) + 0.6 + 0.55}{2 + 1 + 1} = \frac{1.35}{4} = 0.3375$$

so class 0 wins even more clearly (0.6625).

To find good weights, we try them all: each weight from 1 to 3 for each of the three models, 27 combinations, each scored by cross-validation.

Python: Searching over weights.

```
for i in range(1, 4):
    for j in range(1, 4):
        for k in range(1, 4):
            vc = VotingClassifier(estimators=estimators,
                                voting="soft", weights=[i, j, k])
            acc = cross_val_score(vc, X, y, cv=10).mean()
            print(i, j, k, round(acc, 3))
```

The best combination is **(3, 1, 1)** with **0.71**, up from 0.64. It gives logistic regression, the best single model, the biggest say, which makes sense. Trying values and keeping the best is hyperparameter tuning; `GridSearchCV` (the [KNN Note](#), section 4.2) can run the same search for us.

5. One algorithm, different settings

Key point: Instead of picking the best of five SVMs, vote over all five: 0.928, above the best single one (0.894).

Voting does not need different algorithms. It can combine **one algorithm with different hyperparameter values**, and this is often the more useful approach.

Tuning is hard because we do not know which value is right. Take an SVM with a polynomial kernel (the [kernel trick Note](#)) and an unknown degree. On a synthetic dataset of 1,000 rows and 20 columns (`make_classification`), we try degrees 1 to 5, each scored by 10-fold cross-validation:

Degree	1	2	3	4	5	Soft vote of all five
Accuracy	0.851	0.855	0.894	0.854	0.871	0.928

Normally we would keep degree 3. Putting all five SVMs into a soft-voting classifier does better than any of them.

Python: Voting over five SVMs.

```
from sklearn.calibration import CalibratedClassifierCV
from sklearn.svm import SVC

svms = [(f"svm{d}", CalibratedClassifierCV(
    SVC(kernel="poly", degree=d), ensemble=False))
         for d in range(1, 6)]
vc = VotingClassifier(estimators=svms, voting="soft")
cross_val_score(vc, X, y, cv=10).mean()      # 0.928
```

Both approaches are used: different algorithms, or one algorithm with several settings. Neither wins on every dataset, so with ensembles, experimenting matters most.

6. Summary

	Hard voting	Soft voting
Combines	predicted labels	predicted probabilities
Rule	most common label	highest average probability
Needs	<code>predict</code>	<code>predict_proba</code> on every model
Concentric circles (LR, NB, RF)	0.66	0.92
Iris, two species, two columns	0.67	0.64 (0.71 with weights 3, 1, 1)

- `VotingClassifier(estimators=[(name, model), ...], voting="hard" or "soft", weights=[...])`.
- Soft voting is often better and smoother, but not always: treat the voting type as a hyperparameter.
- A vote can lose to its best member; then use that member alone.
- Voting over one algorithm with different settings (five SVM degrees: 0.928) can beat picking the best setting (0.894).

7. Key terms

Term	Meaning
Voting classifier	A classifier that combines several trained classifiers by voting
Hard voting	Predicting the label that most base models predict
Soft voting	Predicting the class with the highest average predicted probability across the base models
weights	<code>VotingClassifier</code> and <code>VotingRegressor</code> setting that gives each base model's vote a different importance
<code>CalibratedClassifierCV</code>	scikit-learn wrapper that gives a classifier, such as an SVM, calibrated probabilities